

Linked List

Linked List

- A **linked list** is a dynamic data structure where each element (called a node) is made up of two items - the data and a reference (or pointer) which points to the next node. A **linked list** is a collection of nodes where each node is connected to the next node through a pointer.

Linked List

```
Struct Node  
{  
    int data;  
    Node* link;  
}
```

```
Node* A;
```

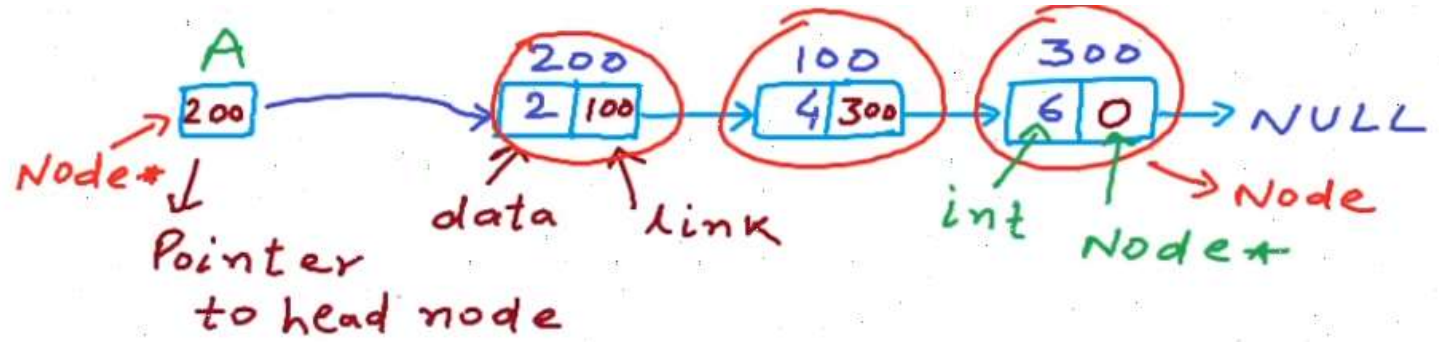
```
A = NULL;
```

```
Node* temp = new Node();
```

```
temp->data = 2;
```

```
temp->link = NULL;
```

```
A = temp;
```



Linked List : Insertion at end

```
Node* A; ✓  
A = NULL; ✓  
Node* temp = new Node();  
temp->data = 2;  
temp->link = NULL;  
A = temp;  
temp = new Node();  
temp->data = 4;  
temp->link = NULL;
```



Traversal

```
Node* temp1 = A;  
while (temp1->link != NULL)  
{  
    temp1 = temp1->link;  
}  
temp1->link = temp;
```

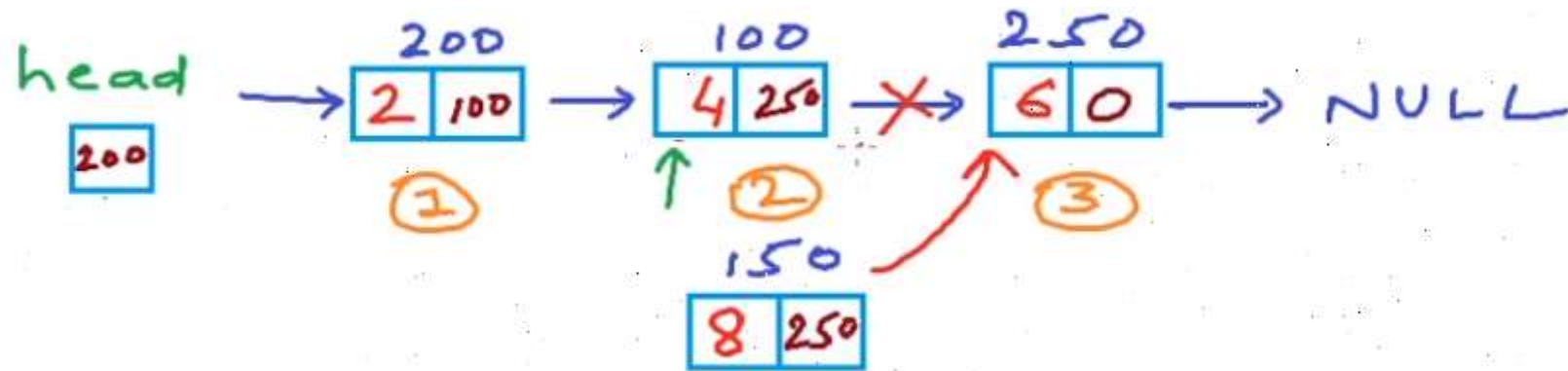
Linked List : Insertion at beginning

```
//Linked List: Inserting a node at beginning
#include<stdlib.h>
#include<stdio.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* head; // global variable, can be accessed anywhere
void Insert(int x);
void Print();
int main() {
    head = NULL; // empty list;
    printf("How many numbers?\n");
    int n,i,x;
    scanf("%d",&n);
    for(i = 0;i<n;i++){
        printf("Enter the number \n");
        scanf("%d",&x);
        Insert(x);
        Print();
    }
}
```

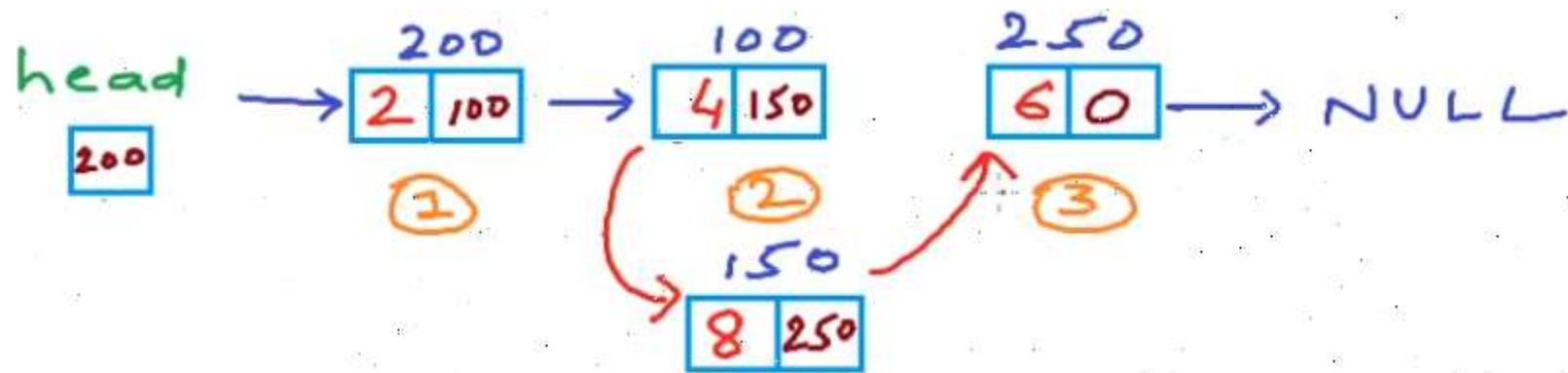
Linked List : Insertion at beginning

```
struct Node* head; // global variable, can be accessed anywhere
void Insert(int x)
{
    struct Node* temp = (Node*)malloc(sizeof(struct Node));
    temp->data = x;
    temp->next = head;
    head = temp;
}
void Print()
{
    struct Node* temp = head;
    printf("List is: ");
    while(temp != NULL)
    {
        printf(" %d", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

Linked List : Insertion at nth position



Linked List : Insertion at nth position



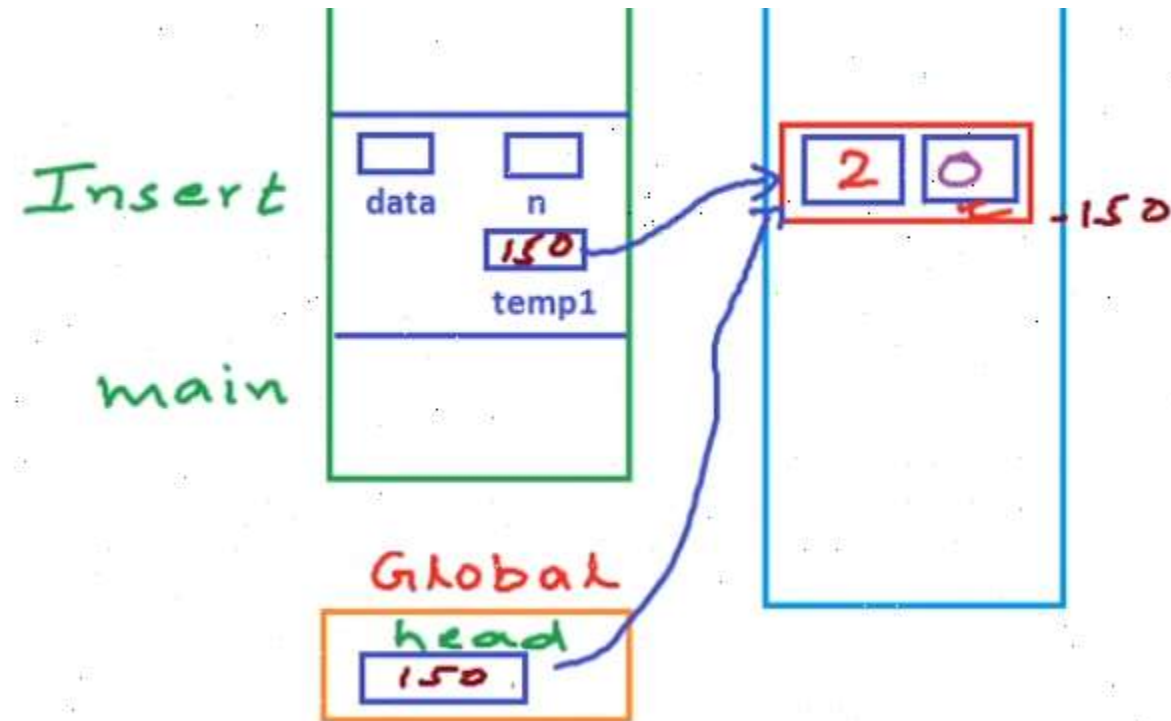
Linked List : Insertion at nth position

```
struct Node {  
    int data;  
    struct Node* next;  
};  
struct Node* head;  
void Insert(int data,int n);  
void Print();  
int main() {  
    head = NULL; //empty list  
    Insert(2,1); //List: 2  
    Insert(3,2); //List: 2,3  
    Insert(4,1); //List: 4,2,3  
    Insert(5,2); //List: 4,5,2,3  
}
```

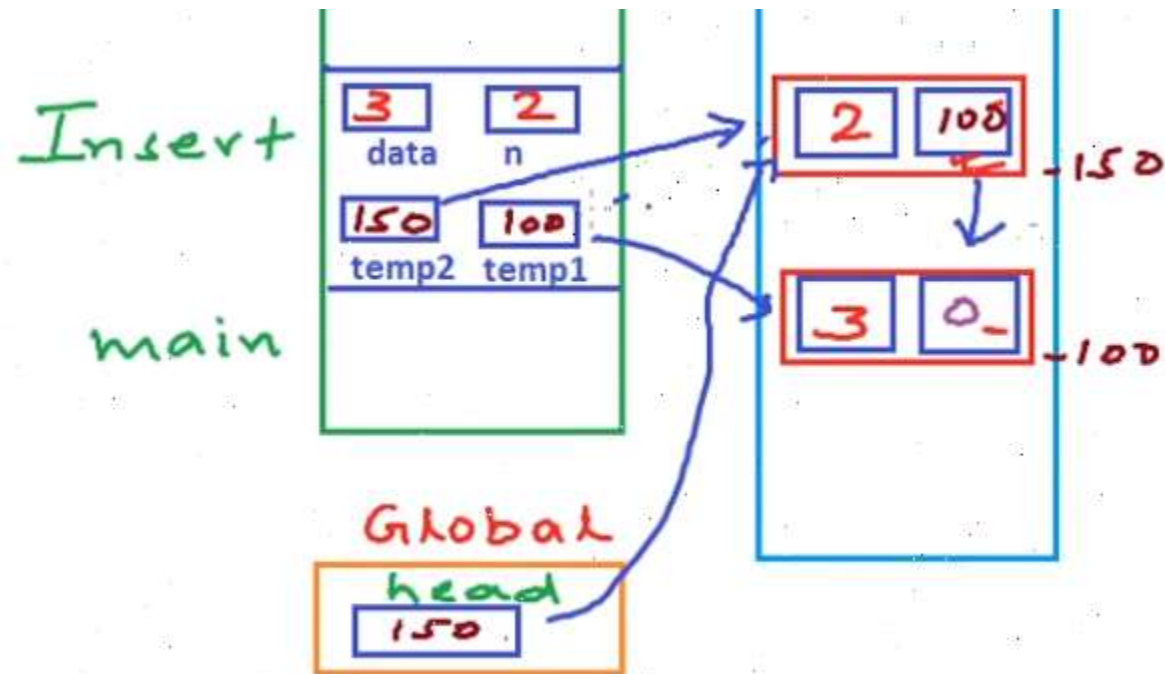
Linked List : Insertion at nth position

```
void Insert(int data,int n) {  
    Node* temp1 = new Node();  
    temp1->data = data;  
    temp1->next = NULL;  
    if(n == 1) {  
        temp1->next = head;  
        head = temp1;  
        return;  
    }  
    Node* temp2 = head;  
    for(int i =0;i<n-2;i++) {  
        temp2 = temp2->next;  
    }  
    temp1->next = temp2->next;  
    temp2->next = temp1;  
}
```

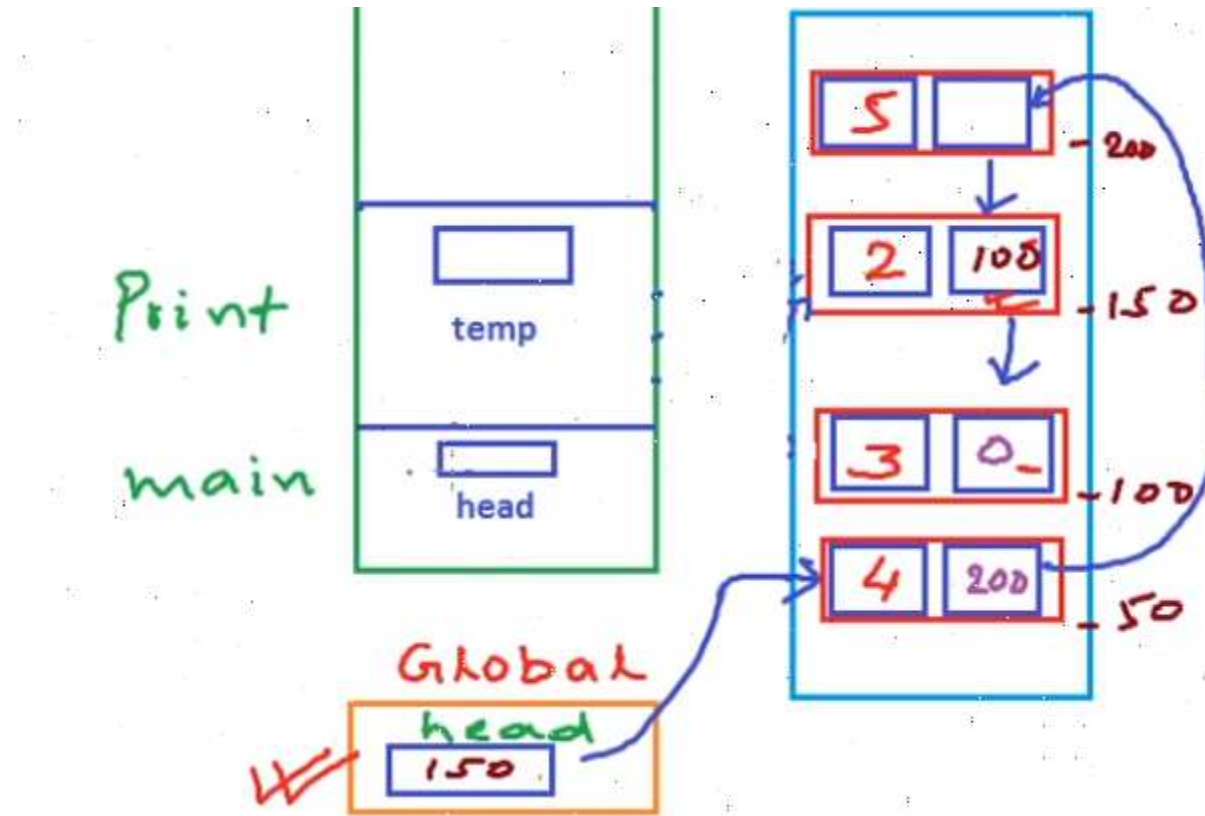
Linked List : Insertion at nth position



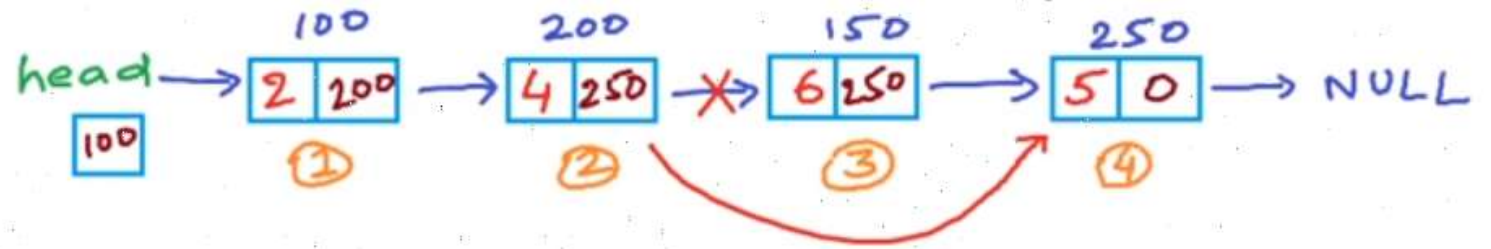
Linked List : Insertion at nth position



Linked List : Insertion at nth position



Linked List : Deletion at nth position



- 1) Fix the links
- 2) Free the space

Linked List : Deletion at nth position

```
//Linked List: Delete a node at nth position
#include<stdio.h>
#include<stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* head; // global
void Insert(int data); // insert an integer at end of list
void Print(); // print all elements in the list
void Delete(int n) // Delete node at position n
int main()
{
}
```

Linked List : Deletion at nth position

```
int main()
{
    head = NULL; // empty list
    Insert(2);
    Insert(4);
    Insert(6);
    Insert(5); //List : 2,4,6,5
    Print();
    int n;
    printf("Enter a position\n");
    scanf("%d",&n);
    Delete(n);
    Print();
}
```

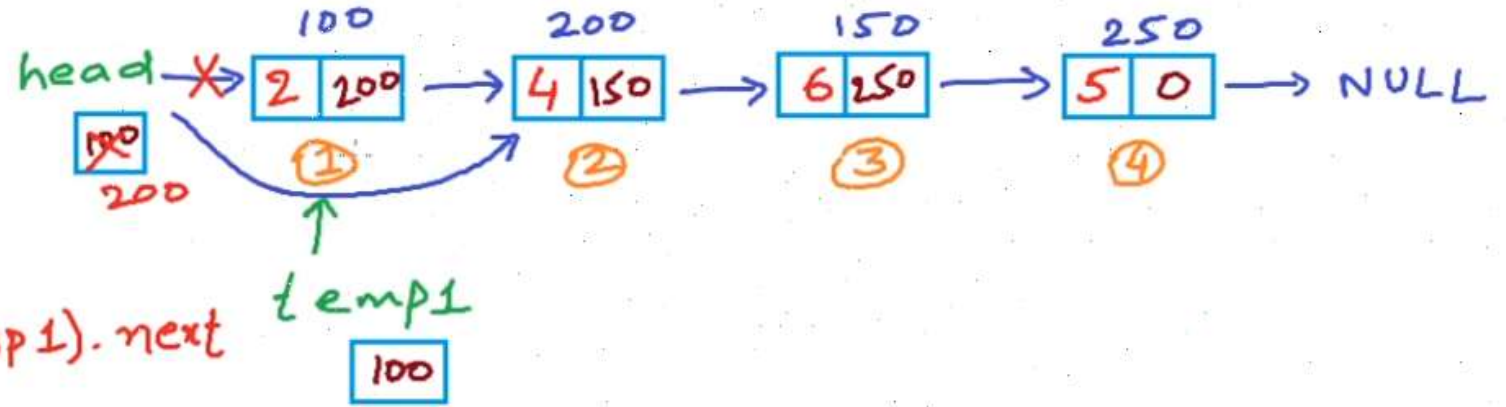

Linked List : Deletion at nth position

```
// Deletes node at position n
void Delete(int n)
{
    struct Node* temp1 = head;
    if(n ==1){
        head = temp1->next; //head now points to second node.
        free(temp1);
        return;
    }
    int i;
    for(i = 0;i<n-2;i++)
        temp1 = temp1->next;
    // temp1 points to (n-1)th Node
    struct Node* temp2 = temp1->next; // nth Node
    temp1->next = temp2->next; // (n+1)th Node
    free(temp2); //delete temp2;
}
```

Linked List : Deletion at nth position

```
void Delete(int n)
{
  ✓✓ struct Node* temp1 = head;
  if(n == 1){
    ✓✓ head = temp1->next;
    ✓✓ free(temp1);
    return;
  }
  int i;
  for(i = 0; i < n-2; i++)
    temp1 = temp1->next;

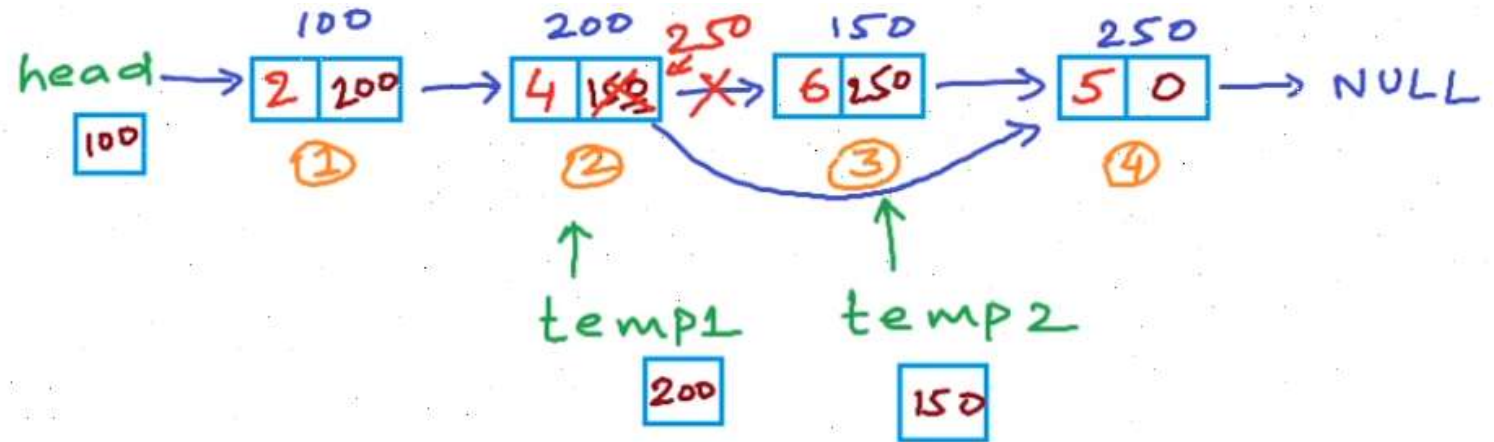
  struct Node* temp2 = temp1->next;
  temp1->next = temp2->next;
  free(temp2);
}
```



Delete(1)

Linked List : Deletion at nth position

```
void Delete(int n)
{
    struct Node* temp1 = head;
    if(n == 1){
        head = temp1->next;
        free(temp1);
        return;
    }
    int i;
    for(i = 0; i < n-2; i++){
        temp1 = temp1->next;
    }
    struct Node* temp2 = temp1->next;
    temp1->next = temp2->next;
    free(temp2);
}
```



Delete(3)